

Matlab Praktikum

Maschinelles Sehen

Prof. Dr.-Ing. Volker Willert

Wintersemester
Übungsblatt 2

In dieser Praktikumseinheit werden folgende Punkte behandelt:

- Ausschnitte von Bildern anzeigen und abspeichern
- Künstliche Testbilder erzeugen
- Koordinaten und *Region of Interest* (ROI) visualisieren
- Gerade in Bild einzeichnen mit Bresenham Algorithmus
- Erstellen eines *High Dynamic Range* (HDR) Farbbildes

Benutzen Sie die auch die Matlab-Hilfe `help` um die Aufgaben zu lösen.

Aufgabe 2.1: Ausschnitte von Bildern anzeigen und abspeichern

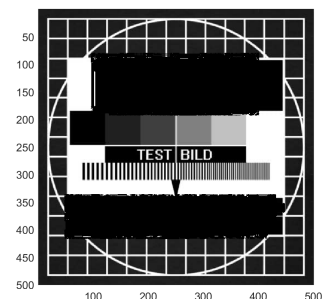
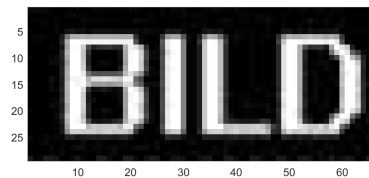
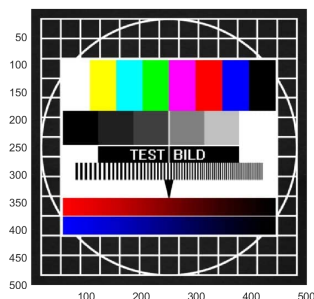


Abbildung 1: Ergebnis 2.1 a) (links), Ergebnis 2.1 b) (mitte), Ergebnis 2.1 c) (rechts)

- Laden Sie das Bild `TestbildFernseher.jpg` in ihren Workspace und zeigen Sie das Bild mit der Funktion `image` und **gleichskalierten Achsen** an (siehe 1, links).
- Benutzen Sie die Werkzeuge `Data Tips` und `Zoom In` im Menü ihrer `Figure`, um die Koordinaten des Bildausschnittes zu ermitteln. Weisen Sie dann diesen Bildausschnitt einem $M \times N \times 3$ - Feld zu und zeigen diesen wieder als Bild an (siehe 1, mitte).
- Setzen Sie alle Pixel des Farbbildes, die keinem Grauwert entsprechen auf die Farbe schwarz und zeigen Sie das Ergebnis an. Hinweis: Ein Grauwert hat in jedem Farbkanal den gleichen Wert, das entspricht jedem Werte-Triple (a, a, a) wobei $a = [0; 255]$. Weiß entspricht z.B. dem Werte-Triple $(255, 255, 255)$ und Schwarz dem Werte-Triple $(0, 0, 0)$. (siehe 1, rechts).

Aufgabe 2.2: Künstliche Testbilder erzeugen

- Erzeugen Sie Bilder der Größe 100×100 Pixel mit zufällig verteilten Grauwerten. Erzeugen Sie zuerst gleichverteilte ganzzahlige Grauwerte mit der Funktion `randi` im Intervall von $[64;255]$ und dann normalverteilte Grauwerte mit der Funktion `randn` (siehe Figure 2). Variieren Sie den Mittelwert und die Varianz und beschreiben Sie den Effekt.
- Erzeugen Sie ein schwarzes Bild mit weißen vertikalen Streifen. Die weißen Streifen und die schwarzen Abstände sollen ihre Breite für jeden neuen Streifen von links nach rechts um 1 erhöhen. Beginnen Sie mit einer Streifenbreite von 1. Das Bild soll eine Größe von 100×110 Pixel haben (siehe Figure 2, rechts).
- Benutzen Sie nun die Funktion `imnoise`, um unterschiedliches Bildrauschen auf dem Testbild 'Auge . jpg' zu simulieren. Erstellen Sie 1) Gaußverteiltes Rauschen, was A/D-Wandlungsfehler simuliert, 2) Poisson Rauschen, was Dunkelstrom simuliert, 3) Salt & Pepper Rauschen, was Pixelfehler simuliert, und 4) Speckle Rauschen, was Bildstörungen von Radaraufnahmen oder Ultraschallbildern simuliert (siehe Figure 3). Benutzen Sie die Funktion `montage`, um die Bilder anzuzeigen.

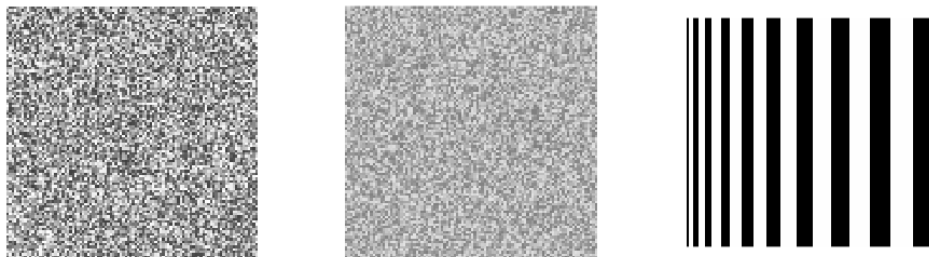


Abbildung 2: Gleichverteiltes Rauschen, normalverteiltes Rauschen, Streifentestbild.

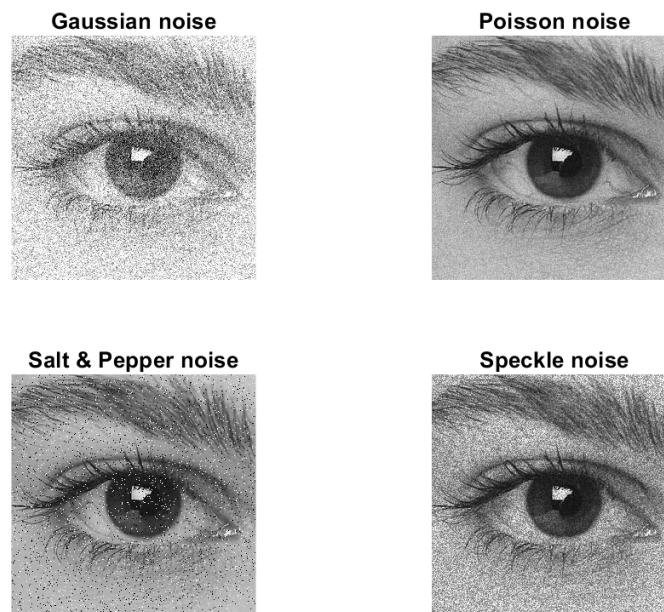


Abbildung 3: 1) Gaußverteiltes Rauschen (oben links), 2) Poisson Rauschen (oben rechts), 3) Salt & Pepper Rauschen (unten links), 4) Specklestörung (unten rechts).

Aufgabe 2.3: Koordinaten und *Region of Interests* (ROIs) visualisieren

- Benutzen Sie die Funktion `insertShape` aus der *Computer Vision Toolbox* und zeichnen Sie eine Gerade, ein Rechteck und einen Kreis in ein Bild. Sie können das Testbild `TestbildFernseher.jpg` verwenden und einige weiße Konturen durch z.B. rote ersetzen.
- Benutzen Sie die Funktion `insertShape`, um eine eigene Funktion `myInsertCross` zu schreiben. Diese Funktion soll an einem Pixel ein Kreuz beliebiger Länge plotten. Folgende Parameter sollen übergeben werden: x -Koordinate, y -Koordinate, Länge l in Pixel.

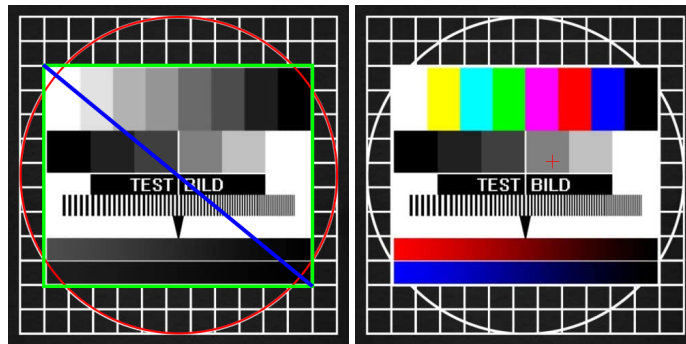


Abbildung 4: Beispiele zur Aufgabe 2.3 a) und 2.3 b).

Aufgabe 2.4: Gerade in Bild einzeichnen mit Bresenham Algorithmus

Der Bresenham Algorithmus ist ein einfacher Algorithmus, um einfache Geometrien, wie Geraden und Kreise in ein gerastertes Bild zu zeichnen. Er ist sehr recheneffizient, da er ohne Multiplikation, Division und Gleitkommazahlen auskommt. Er wird beispielsweise auf Wikipedia beschrieben: <https://de.wikipedia.org/wiki/Bresenham-Algorithmus>.

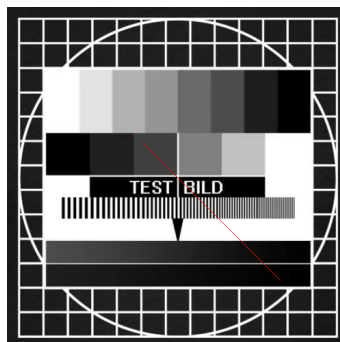


Abbildung 5: Rote Linie mit Bresenham Algorithmus erstellt.

- Schreiben Sie eine Funktion `function [xCoords yCoords]=myBresenham(xStart,yStart,xEnd,yEnd)`, welche die Koordinaten einer Linie in den Vektoren `xCoords` und `yCoords` abspeichert. Die Linie beginnt am Punkt `(xStart, yStart)` und endet am Punkt `(xEnd, yEnd)`.
- Visualisieren Sie das Ergebnis, indem Sie an den berechneten Koordinaten die Farbe des Bildes auf rot setzen und zeigen Sie das Ergebnis an (siehe Figure 5).

Aufgabe 2.5: Erstellen von HDR Bildern



Abbildung 6: Bilderreihe mit unterschiedlichen Belichtungszeiten von normalen *Low Dynamic Range* (LDR) Bildern

High Dynamic Range (HDR) Bilder dienen dazu den Kontrast in einer schlecht beleuchteten Szene zu erhöhen. Dazu wird eine Bilderreihe mit unterschiedlichen Belichtungszeiten von normalen *Low Dynamic Range* (LDR) Bildern aufgenommen. In jedem dieser LDR Bilder sind dann je nach Beleuchtungsverhältnis bestimmte Bereiche gut belichtet, einige unterbelichtet und einige überbelichtet. Diese können dann zu einem einzigen HDR Bild umgerechnet werden, in dem für alle Bereiche die bestmögliche Belichtung vorherrscht. Abbildung 6 zeigt eine Abfolge einer Belichtungsserie von LDR Webcam-Bildern mit unterschiedlichen Belichtungszeiten. Schreiben Sie nun ein kleines Programm, um Bilder einer Webcam mit unterschiedlichen Belichtungszeiten (Exposure) zu erzeugen. Benutzen Sie dann die Funktion `makehdr`, um ein HDR Bild aus der Bilderreihe zu erzeugen und mappen Sie dann das HDR Bild mit der Funktion `tonemap` auf den normalen RGB Farbraum. Falls Sie keine Belichtungszeit an ihrer Webcam einstellen können, dann können Sie die Testbilder `Exposure_4.jpg` - `Exposure_9.jpg` verwenden.



Abbildung 7: High Dynamic Range Ergebnis.